A Project Report on

Hate Speech Detection

A Dissertation submitted to JNTU Hyderabad in partial fulfillment of the academic requirements for the award of the degree.

Bachelor of Technology

In

Computer Science and Engineering

Submitted by

D.V.Bhuvaneswar reddy
(20H51A05D9)

Under the esteemed guidance of
Ms. E. Krishnaveni
(Assistant Professor)

EXPLORE TO INVENT

Department of Computer Science and Engineering

CMR COLLEGE OF ENGINEERING & TECHNOLOGY
(UGC Autonomous)
*Approved by AICTE *Affiliated to JNTUH *NAAC Accredited with A^+ Grade
KANDLAKOYA, MEDCHAL ROAD, HYDERABAD - 501401.

2020- 2024

CMR COLLEGE OF ENGINEERING & TECHNOLOGY

KANDLAKOYA, MEDCHAL ROAD, HYDERABAD – 501401

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Major Project report entitled **"Hate Speech Detection"** being submitted by D.V.Bhuvaneswar reddy (20H51A05D9), in partial fulfillment for the award of **Bachelor of Technology** in **Computer Science and Engineering** is a record of bonafide work carried out under my guidance and supervision.

The results embodies in this project report have not been submitted to any other University or Institute for the award of any Degree.

Ms. E. Krishnaveni **Assistant Professor** **Dept. Of CSE**	**Dr. S. Siva Skandha Sanagala** **Associate Professor and HOD** **Dept. of CSE**	**EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

With great pleasure I want to take this opportunity to express my heartfelt gratitude to all the people who helped in making this project a grand success.

I am grateful to **Ms. E. Krisnaveni, Assistant Professor**, Department of Computer Science and Engineering for her valuable technical suggestions and guidance during the execution of this project work.

I would like to thank**, Dr. Siva Skandha Sanagala,** Head of the Department of Computer Science and Engineering, CMR College of Engineering and Technology, who is the major driving forces to complete our project work successfully.

I am very grateful to **Dr. Ghanta Devadasu**, Dean-Academics, CMR College of Engineering and Technology, for his constant support and motivation in carrying out the project work successfully.

I am highly indebted to **Major Dr. V A Narayana,** Principal, CMR College of Engineering and Technology, for giving permission to carry out this project in a successful and fruitful way.

I would like to thank the **Teaching & Non- teaching** staff of Department of Computer Science and Engineering for their co-operation.

I express my sincere thanks to **Shri. Ch. Gopal Reddy**, Secretary& Correspondent, CMR Group of Institutions, and **Shri Ch Abhinav Reddy**, CEO, CMR Group of Institutions for their continuous care and support.

Finally, I extend thanks to my parents who stood behind me at different stages of this Project. I sincerely acknowledge and thank all those who gave support directly or indirectly in completion of this project work.

D.V.Bhuvanesar reddy 20H51A05D9

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	LIST OF FIGURES	iii
	ABSTRACT	iv
1	**INTRODUCTION**	1
	1.1 Problem Statement	2
	1.2 Research Objective	3
	1.3 Project Scope and Limitations	4
2	**BACKGROUND WORK**	6
	2.1. Hate speech detection using Machine Learning	7
	2.1.1. Introduction	7
	2.1.2. Merits, Demerits and Challenges	8
	2.1.3. Implementation of Hate speech detection	10
	2.2. Hate speech detection in Indonesian language on Instagram	12
	2.2.1. Introduction	12
	2.2.2. Merits, Demerits and Challenges	13
	2.2.3. Implementation in Indonesian language	15
	2.3. Automated hate speech detection on Twitter	17
	2.3.1. Introduction	17
	2.3.2. Merits, Demerits and Challenges	18
	2.3.3. Implementation of Automated hate on Twitter	20
3	**PROPOSED SYSTEM**	22
	3.1. Objective of Proposed Model	23
	3.2. Algorithms Used for Proposed Model	24
	3.3. Designing	29
	3.3.1.UML Diagram	30
	3.4. Stepwise Implementation and Code	35
4	**RESULTS AND DISCUSSION**	43
	4.1 Results	44
	4.2 Performance metrics	46

5 CONCLUSION 47

5.1 Conclusion and Future Enhancement 48

REFERENCES 50

GITHUB LINK 52

PUBLISHED PAPER & CERTIFICATES 53

List of Figures

FIGURE NO.	TITLE	PAGE NO.
2.1.3	Architecture Using Machine learning	11
2.2.3	Indonesian language on Instagram	24
2.3.3	Automated hate speech detection on Twitter	25
3.3	Flow diagram of Proposed model	40
3.3.1	UML Diagram	41
3.3.3	Sequence Diagram	42
3.3.4	Collaboration	43
4.1.1	Implementation Interface in google colab	44
4.1.2	Output Interface	44
4.1.3	Comment Section In Output Phase	45
4.1.4	Final Output	45
4.1.5	Model Learning Curve	46

ABSTRACT

Hate speech is a pervasive and harmful problem in online communication, posing serious threats to social cohesion, individual well-being, and public discourse. This project focuses on the development and implementation of a robust hate speech detection system using advanced Natural Language Processing (NLP) techniques. The objective is to create a tool that can automatically identify and classify hate speech content in text data, enabling a proactive approach to combat online hate speech.

The proliferation of hate speech online has exacerbated societal divisions and contributed to the spread of misinformation and discrimination. Traditional methods of moderating online content often fall short due to the sheer volume of data generated daily. By leveraging cutting-edge NLP algorithms, this project aims to enhance the efficiency and accuracy of hate speech detection, enabling platforms to swiftly identify and mitigate harmful content. By doing so, we can create safer online environments conducive to constructive dialogue and respectful discourse.

Moreover, the development of a robust hate speech detection system aligns with broader efforts to uphold fundamental human rights, including the right to freedom of expression without fear of persecution or discrimination. By proactively identifying and addressing hate speech, we can promote inclusivity, tolerance, and mutual respect in online communities. Through collaboration with stakeholders across academia, industry, and civil society, this project seeks to foster interdisciplinary dialogue and collective action to address the complex challenges posed by online hate speech.

.

CHAPTER 1
INTRODUCTION

CHAPTER 1
INTRODUCTION

1.1. Problem Statement

The number of people using online forums and social networks has grown exponentially in the last century. The number of active social media users as of March 2023 is 4.95 billion. Three of the most widely used micro blogging platforms for communication are Twitter, Facebook, and Instagram. Twitter generates 350000 tweets every second. People with varying backgrounds in culture and education share their opinions on daily topics such as political opinions, product reviews, movie reviews, and so forth. However, because of disagreements in viewpoint, there are instances when conversations between individuals devolve into hostile language. Bullying is one of the main causes of suicide, yet such language can take many different forms. Innocent victims of mob lynchings perish in large numbers every year. Since identifying hate speech on these kinds of websites by hand is an extremely laborious and time consuming process, this has led to the development of a model that can do so automatically. The use of such terminology must end immediately, as must its dissemination. The term "hate speech" refers to any expression that disparages an individual or group based on characteristics including race, religion, ethnicity, gender, handicap, or gender identity.

1.2. Research Objective

The primary research objective of this project is to pioneer the development of a sophisticated hate speech detection system leveraging advanced natural language processing (NLP) techniques. This system aims to transcend the limitations of existing approaches by offering a comprehensive solution capable of swiftly and accurately identifying hate speech across diverse online platforms. By harnessing the power of machine learning algorithms and linguistic analysis, the system will be designed to discern subtle nuances and contextual cues inherent in hate speech, thereby enhancing its efficacy in distinguishing between genuinely harmful content and benign expressions of opinion.

Furthermore, the envisioned hate speech detection system seeks to address the evolving nature of online discourse by remaining adaptable and responsive to emerging trends and tactics employed by purveyors of hate speech. Through continuous refinement and optimization, the system aims to stay ahead of evolving linguistic patterns and adversarial strategies, ensuring its effectiveness in combatting hate speech across various digital environments. By fostering interdisciplinary collaboration and drawing upon insights from fields such as psychology, sociology, and linguistics, this project endeavors to develop a multifaceted approach to hate speech detection that not only improves algorithmic accuracy but also promotes a deeper understanding of the underlying social dynamics driving the propagation of hateful rhetoric online.

1.3. Project Scope and Limitations:

Project Scope:

- Data Collection and Annotation: The project entails sourcing a diverse and representative dataset containing examples of hate speech across different online platforms. Each data point will undergo meticulous annotation to categorize instances of hate speech accurately, laying the foundation for robust model training.
- Text Data Preprocessing and Cleaning: Prior to model training, extensive preprocessing and cleaning of the text data will be conducted to standardize formats, remove noise, and mitigate biases. This step is crucial for ensuring the quality and reliability of the dataset, thereby enhancing the performance of the hate speech detection system.
- NLP Model Selection and Fine-Tuning: The project involves evaluating various NLP models to identify the most suitable architecture for hate speech detection. Following model selection, iterative fine-tuning will be performed to optimize performance on the annotated dataset, striking a balance between precision and recall.
- Performance Evaluation Metrics: The effectiveness of the hate speech detection system will be assessed using a range of performance metrics, including accuracy, precision, recall, and F1-score. Additionally, considerations of fairness and bias mitigation will be integrated into the evaluation process to ensure equitable outcomes across diverse demographic groups.
- Deployment and Integration: Upon successful model development, the hate speech detection system will be seamlessly integrated into the target platform, whether it's a social media network, online forum, or comment section. Continuous monitoring mechanisms will be implemented to track model performance and facilitate timely updates in response to evolving hate speech trends.

Limitations:

- Data Bias and Representativeness: Despite efforts to collect a representative dataset, inherent biases may still exist, leading to underrepresentation or misrepresentation of certain demographics or linguistic variations. This could limit the generalizability of the hate speech detection system and introduce disparities in its performance across different groups.

- Annotator Subjectivity: The process of annotating hate speech data relies on human judgment, which can be subjective and influenced by individual biases. Inconsistent annotations may introduce noise and inaccuracies into the training dataset, impacting the effectiveness of the NLP model.

- Complexity of Hate Speech: Hate speech is often nuanced and context-dependent, making it challenging to capture and classify accurately using automated methods. The inability of NLP models to fully grasp linguistic subtleties and cultural nuances may result in false positives or false negatives, compromising the reliability of the detection system.

- Limited Scope of Metrics: While accuracy and other traditional evaluation metrics provide valuable insights into model performance, they may not capture the full spectrum of outcomes relevant to hate speech detection. Metrics such as fairness, interpretability, and contextual appropriateness are essential but may be difficult to quantify or evaluate systematically.

- Deployment Challenges: Integrating the hate speech detection system into diverse online platforms poses technical and logistical challenges. Compatibility issues, resource constraints, and resistance from platform owners or users may hinder the seamless deployment and adoption of the system, limiting its impact and effectiveness.

CHAPTER 2
BACKGROUND WORK

CHAPTER 2

BACKGROUND WORK

2.1.1 Hate speech detection using machine learning

2.1.1. Introduction

Hate speech, encompassing discriminatory language targeting individuals or groups based on various attributes, presents a pervasive challenge within online social networks, raising concerns about its potential to incite violence and perpetuate societal divisions [1]. Addressing this issue necessitates effective detection strategies tailored to the dynamic nature of online platforms. Machine learning, particularly deep learning methodologies, has emerged as a promising avenue for combating hate speech [2]. Leveraging large datasets and advanced algorithms, machine learning techniques enable the automated identification and classification of hate speech content [2, 9]. Studies have explored diverse approaches, including token replacement-based data augmentation methods and sentiment analysis, to enhance hate speech detection accuracy [2, 9].

One prominent approach involves the utilization of deep learning technologies, such as recurrent neural networks (RNNs) and convolutional neural networks (CNNs), to analyze textual content and identify instances of hate speech [1, 4]. Additionally, techniques like word embeddings have proven effective in capturing semantic nuances, thereby improving detection precision [4]. Efforts have also focused on addressing specific challenges, such as developing multilingual detection models and creating interfaces tailored to different linguistic and cultural contexts [3]. Despite notable progress, challenges persist, including the need for robust training data and ethical considerations surrounding algorithmic content moderation [9]. Collaborative interdisciplinary research endeavors remain crucial for advancing the state-of-the-art in hate speech detection and mitigation [1, 3, 9].

2.1.2. Merits, Demerits and Challenges

Merits:

- Machine learning allows for the automated detection of hate speech, which can be difficult and time-consuming to do manually, especially on a large scale.
- Machine learning algorithms provide consistent and unbiased evaluations, as they do not suffer from fatigue or personal biases as human moderators might.
- Automated detection can free up human moderators to focus on more nuanced and context specific cases, improving the overall efficiency of content moderation

Demerits:

- ML models can inherit biases present in their training data, potentially leading to incorrect identification or over-censorship of content. Ensuring fairness is a major challenge.
- Analyzing user-generated content raises privacy concerns, and users may feel uncomfortable knowing their messages are being analyzed by algorithms.
- decisions they make can be opaque and difficult for humans to interpret why the decision was made

.

Challenges:

- Biased Training Data: Sometimes, the information used to teach computers about hate speech can be unfair because it reflects existing prejudices or opinions of the people who labeled it. This can make the computer models not very accurate and might end up wrongly flagging or blocking content.

- Privacy Concerns: When computers look at what people say online to find hate speech, it can feel like an invasion of privacy to those people. They might worry about their messages being watched and feel less comfortable using the platform. It's hard to balance keeping things safe and respecting people's privacy.

- Opacity of Decision-Making: Sometimes, the way computers decide if something is hate speech can be a bit mysterious. This makes it hard for people to understand why certain things are flagged or removed, which can lead to questions about whether the process is fair and transparent.

- Dynamic Nature of Hate Speech: Hate speech changes a lot and depends on what's happening in the world. Computers might struggle to keep up with these changes because they learn from fixed examples.

- Scalability and Resource Constraints: Making sure computers can quickly and accurately spot hate speech in all the messages people post online requires a lot of computer power and resources.

2.1.3. Implementation of Hate Speech Detection

Detecting hate speech using machine learning algorithms such as Support Vector Machines (SVM) and Naive Bayes is a common approach in natural language processing. Here are some steps you can take to create a hate speech detection system using these algorithms.

1. **Collect a Hate Speech Dataset:**

- Select a suitable hate speech dataset such as the Hate Speech and Offensive Language dataset or the Twitter Hate Speech dataset.
- Load the dataset into your programming environment. Ensure that it includes labeled examples of hate speech and non-hate speech.

2. **Pre-process the Data:**

- Perform text pre-processing steps to clean and prepare the raw text data.
- Split the text into individual words or tokens.
- Eliminate common words like "the," "is," and "and" that don't carry much meaning.
- Stemming or Lemmatization: Reduce words to their root form (e.g., "running" becomes "run").
- Convert text to lowercase to ensure consistency.

3. **Feature Extraction:**

- Utilize various techniques to extract features from the pre-processed text.
- Bag of Words (BoW): Represent each document as a vector of word counts.
- Term Frequency-Inverse Document Frequency (TF-IDF): Weigh the importance of words based on their frequency in the document and across the entire dataset.
- Word Embeddings: Represent words as dense vectors in a continuous space, capturing semantic relationships.

4. **Train the Model:**

- Split the dataset into training and validation sets to assess model performance.
- Initialize SVM and Naive Bayes classifiers using libraries like scikit-learn in Python.
- Tune hyperparameters (e.g., regularization parameter for SVM, smoothing parameter for Naive Bayes) using cross-validation to optimize model performance.

5. **Evaluate and Test the Model:**

- Assess the performance of the trained models on the validation set using evaluation metrics such as accuracy, precision, recall, and F1-score.
- Test the models on unseen data (e.g., a separate test dataset or real-world data) to evaluate generalization performance.
- Fine-tune the models as needed based on evaluation results and iterate on the training process if necessary.

6. **Deployment and Monitoring:**

- Deploy the trained models into a production environment or integrate them into your application.
- Monitor model performance over time and update them periodically to adapt to evolving trends in hate speech.
- Implement mechanisms for user feedback and reporting to improve the robustness and effectiveness of the hate speech detection system.

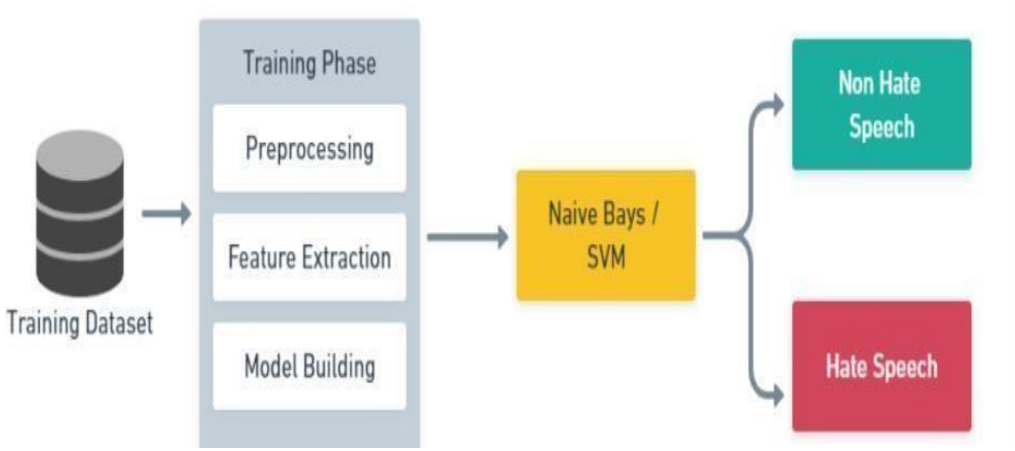


Fig.no.2.1.3: Architecture of Using Machine learning

2.2. Hate Speech Detection in Indonesian language on Instagram Comment Section

2.2.1. Introduction

In this modern era, technologies development has created a new platform in the way we share our thoughts. According to a thesis by Paula Fortuna in 2017 about hate speech, hate speech is language that attacks or diminishes, that incites violence or hate against groups, based on specific characteristics such as physical appearance, religion, descent, national or ethnic origin, sexual orientation, gender identity or other, and it can occur with different linguistic styles, even in subtle forms or when humor is used[1]. Indonesia has its own definition of hate speech. Indonesian ITE constitution section 28 subsection 2 that regulate about hate speech in social media said: "People who spread hate speech are every person thatdeliberately and without right spread information that causes hatred among an individual/ certain group of people based on ethnic, race, religion, and class". This constitution rounds up many people because of the increasing number of hate speech on social media. Opinions in a form of hate speech are one of the problems that authorities find hard to solve, because of its number and variations. Hate speech in media social can be found in many forms, such as comments, pictures, or videos. Hate speech cases number has been increasing rapidly in Indonesia, especially since the election of Jakarta Governor in 2018 and Presidency Election in 2019. Minister of Technology Communication and Informatics Rudiantara said that socialmedia like Instagram and Facebook had become the source of hate speech such as radicalism, terrorism, and extremism.

2.2.2. Merits, Demerits and Challenges

Merits:

- Deep learning models can handle text data without extensive preprocessing, which can save time and resources in building a hate speech detection system.
- An effective deep learning-based system can significantly reduce the burden on human moderators, allowing them to focus on more nuanced cases and ensuring that hate speech is quickly addressed.

Demerits:

- Deep learning models are heavily reliant on the quality and diversity of the training data.
- Hate speech evolves, and the model's performance can degrade over time if not continuously updated and retrained. This requires ongoing resources and attention.

Challenges:

- **Language Complexity:** Indonesian language has various dialects, slangs, and informal expressions which can be challenging to identify and categorize accurately. Moreover, hate speech can be subtle and implicit, making it harder to detect without context.

- **Contextual Understanding:** Understanding the context of a conversation is crucial in identifying hate speech. Words or phrases that may seem innocuous in one context could be offensive or hateful in another. Contextual nuances are particularly pronounced in social media comments, where conversations can be fragmented and lack clear context.

- **Data Imbalance:** Hate speech data is often imbalanced, with a significant majority of comments being non-offensive. This can lead to biases in the model's predictions, where it may inaccurately classify most comments as non-offensive.

- **Evolution of Language and Slang:** Language is constantly evolving, especially on social media platforms. New slangs, memes, and expressions emerge frequently, making it challenging for models trained on static datasets to adapt to these changes.

- **Multimodal Content:** Instagram is a visual-centric platform, where images and videos are as important as text. Hate speech can also manifest through images, emojis, or videos, which adds complexity to the detection task. Integrating multimodal features into the detection system is necessary for comprehensive coverage.

2.2.3. Implementation of Hate Speech Detection in Indonesian language on Instagram Comment Section

Gather a labeled dataset of Instagram comments in Indonesian. You'll need examples of both hate speech and non-hate speech comments.

- Annotate the comments as either "hate speech" or "not hate speech."

Clean the comments by removing special characters, emojis, and irrelevant information.

- Tokenize the comments into words or subwords.
- Convert the text data into numerical representations (e.g., word embeddings).

Build a DNN model for hate speech detection. Common architectures include:

- Recurrent Neural Networks (RNNs): Good for sequential data like text.
- Convolutional Neural Networks (CNNs): Useful for capturing local patterns.
- Transformer-based models**: Effective for understanding context.
- Add layers (e.g., embedding layer, hidden layers, output layer).
- Use activation functions (e.g., ReLU, sigmoid)

Split your dataset into training, validation, and test sets. Train the model using the training data Optimize hyperparameters (learning rate, batch size, etc.). Monitor loss and accuracy during training.

- Evaluate the model on the validation set to prevent overfitting.
- Metrics to consider: accuracy, precision, recall, F1-score.
- Adjust the model architecture or hyperparameters if needed.

Once satisfied with the model's performance, deploy it to your Instagram comment section. Monitor the system in real-time and fine-tune as necessary. When a new comment is posted, preprocess it and pass it through the trained model. The model will predict whether the comment contains hate speech or not.

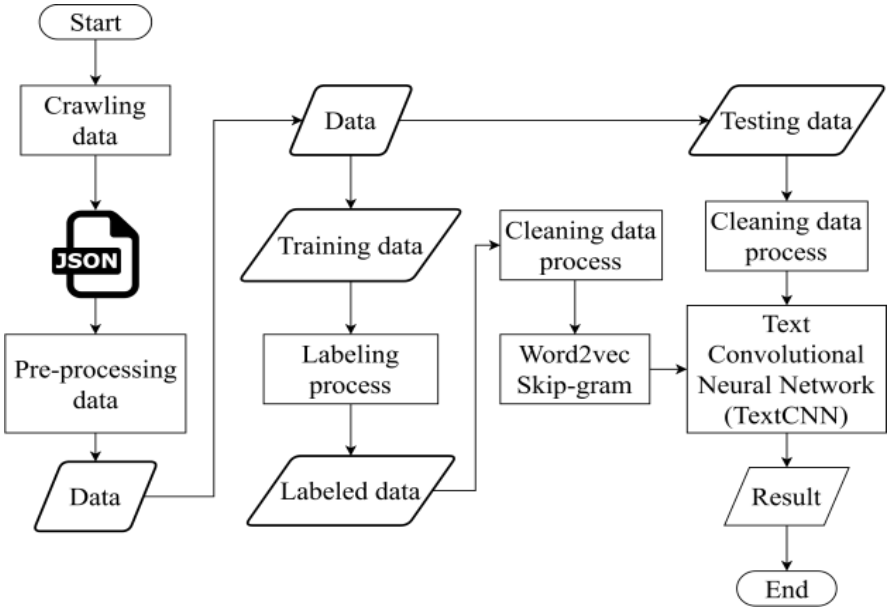


Fig.no.2.2.3: Proposed system of Indonesian language on Instagram

A dataset of Instagram comments is crawled by using hashtags which possible contain hate speech and Instagram scraper3. The Instagram dataset is stored in a JSON file, including an image URL, caption, posting time, comments, et cetera. To extract comments from a JSON file, we use an algorithm4 to transfer it into a table. The dataset is divided into 2 types of data, namely training data with ratio of 80% and testing data with ratio of 20%. The training data are labeled by two annotators, which is the author and Indonesian language expert, by referring to the proposed Indonesian hate as a guidance.

2.3. Automated Hate Speech Detection on Twitter:

2.3.1. Introduction

Hate speech, defined as any form of communication that disparages a person or a group based on characteristics such as race, religion, ethnicity, sexual orientation, disability, or gender, poses significant challenges in online social networks [1]. The proliferation of hate speech online has led to increased concerns about its societal impact, including its potential to incite violence, perpetuate discrimination, and undermine social cohesion. Addressing this issue requires effective detection and mitigation strategies tailored to the dynamics of online platforms.

Machine learning, particularly deep learning techniques, has emerged as a promising approach for detecting hate speech in social networks. By leveraging large datasets of labeled examples, machine learning algorithms can learn patterns and characteristics associated with hate speech, enabling automated identification and classification of problematic content [2]. Various studies have explored the application of machine learning in hate speech detection, employing techniques such as token replacement-based data augmentation [2], sentiment analysis, and natural language processing [9]. One notable approach involves the development of hate speech detection systems using deep learning technologies. These systems typically utilize neural network architectures, such as recurrent neural networks (RNNs) and convolutional neural networks (CNNs), to analyze textual content and identify instances of hate speech [1]. Additionally, techniques such as word embeddings, which represent words as dense vectors in a continuous semantic space, have been instrumental in capturing nuanced semantic relationships and improving detection accuracy [4].

2.3.2. Merits, Demerits and Challenges

Merits:

Tailored Solutions: Acknowledges the unique challenges of online social networks, emphasizing the need for specialized approaches to address platform dynamics effectively.

Big Data Utilization: Recognizes social media content as valuable big data, allowing for comprehensive analysis of user opinions and behaviors, providing valuable insights into social trends and dynamics.

Real-time Insights: Offers the capability to analyze real-time data streams from online social networks, enabling prompt responses to emerging trends or issues.

Demerits:

Content Control Challenges: Faces difficulty in controlling vast amounts of content, particularly in moderating harmful or inappropriate language, posing risks to user experience and community well-being.

Cultural and Linguistic Diversity: Deals with the complexities of diverse user backgrounds and languages, leading to challenges in accurately interpreting and managing language nuances and cultural contexts, potentially resulting in biased or misinterpreted analyses.

Challenges:

- **Data Collection:** Ensuring a diverse dataset involves navigating biases in data collection methods, while representative sampling requires addressing ethical concerns regarding the handling of potentially harmful content.

- **Labeling for Supervised Learning:** Consistent and accurate labeling necessitates meticulous human annotation to overcome subjectivity, with attention to nuanced language and cultural differences complicating the task.

- **Text Data Preprocessing:** Developing robust preprocessing techniques entails devising methods to handle varied text formats efficiently, while ensuring irrelevant information removal does not inadvertently distort context.

- **Text Normalization:** Addressing language-specific nuances in normalization methods requires careful consideration of dialects and linguistic variations to ensure accurate representation of meaning.

- **Model Deployment and Integration:** Ensuring seamless integration necessitates compatibility with existing platforms and robust infrastructure, with performance optimization being crucial for real-time deployment.

- **Ethical Considerations:** Managing biases in model predictions involves ongoing monitoring and refinement to mitigate unintended consequences, while safeguarding user privacy and upholding principles of free speech requires a delicate balance between content moderation and user rights.

2.3.3. Implementation of Automated Hate Speech Detection on Twitter

Detecting and Monitoring Hate Speech in Twitter:

Key Contributions:

Social Network Analysis: social network analysis techniques to visualize hate speech trends.

Novel Dataset: It introduces a novel public dataset containing 6,000 expert-labeled tweets in Spanish.

Classification Approach: Various classification approaches are compared, with the best model achieving an area under the curve (AUC) of 0.828 on their dataset.

The winning approach combines a LTSM+MLP neural network that considers tweet content (words, emojis, expressions) enriched by tf-idf features.

Keywords: hate crime, sentiment analysis, text classification, predictive policing, social network analysis.

Detecting Hate Speech on Twitter Network Using Ensemble Machine Learning:

This study proposes a voting ensemble machine learning method that combines the strengths of logistic regression, decision trees, and support vector machines.

The goal is to automatically detect hate speech in tweets.

Detecting Hate Speech and Offensive Language on Twitter using Machine Learning:

approach to classify tweets into three classes: hateful, offensive, and clean.

They experiment with n-grams as features and use **TF-IDF values for multiple machine learning models.

Hate Speech Detection on Twitter Using Machine Learning:

This paper suggests a method for identifying hate speech on Twitter.

It relies on automatically gathered unigrams and patterns from the training set, which are then used to train a machine learning algorithm.

Effective Hate-Speech Detection in Twitter Data Using Recurrent Neural Networks:

classifier for detecting hate speech in short text (such as tweets).

Their classifier incorporates features related to users' behavioral characteristics, including their tendency to post abusive messages.

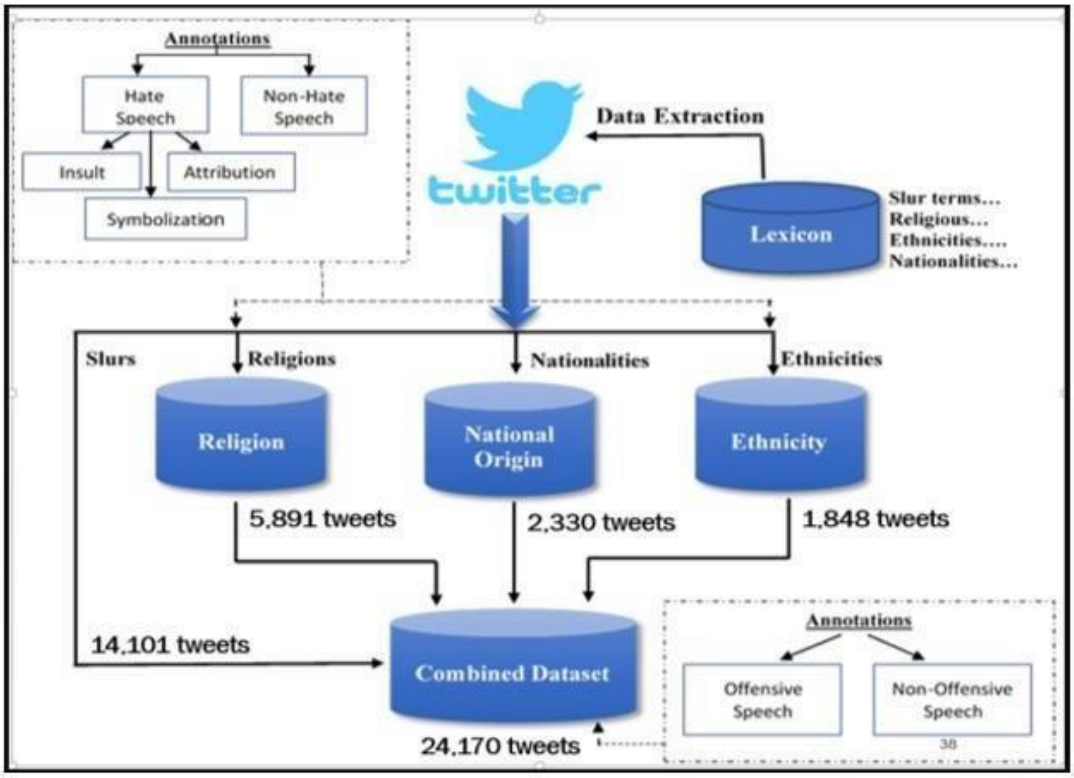


Fig 2.3.3 methodology of automated hate speech detection on Twitter.

CHAPTER 3
PROPOSED SYSTEM

CHAPTER 3

PROPOSED SYSTEM

3.1. Objective of proposed model:

The objective of hate speech detection using Deep Neural Network (DNN) algorithms is to automatically and accurately identify instances of hate speech or abusive language in text data, particularly in dynamic online environments like social media platforms. Leveraging the power of DNNs, these systems aim to scale effectively to process large volumes of user-generated content while adapting to the variability and complexity of languages such as Indonesian. By learning intricate linguistic patterns and contextual cues from raw text data, DNN algorithms facilitate real-time detection, enabling timely intervention to mitigate the spread of hateful content. Moreover, continuous learning and improvement mechanisms ensure that the hate speech detection systems remain effective and up-to-date, capable of addressing evolving language trends and emerging forms of abuse. Ultimately, the objective is to develop robust, interpretable, and scalable solutions that foster a safer and more inclusive online environment, while upholding principles of fairness, transparency, and user privacy.

By the end of this research, we aim to present a comprehensive understanding of our Hate Speech Detection Model, its performance metrics, and its potential impact on fostering a safer and more inclusive online environment. As the digital realm continues to shape the contours of our societal interactions, the need for effective mechanisms to counteract hate speech becomes increasingly urgent. Through this research, we aspire to contribute to the collective pursuit of fostering online spaces where diverse voices can coexist harmoniously, free from the shadows of hate and discrimination.

3.2. Algorithms used for proposed model:

Text Vectorization in Hate Speech Detection:

In hate speech detection, text data must be converted into numerical format for deep learning models to process.

Tokenization: For example, the sentence "You are a worthless loser" would be tokenized into the following tokens: ["You", "are", "a", "worthless", "loser"].

Each unique token is assigned a unique integer index. For example, the vocabulary for the above sentence might look like this: {"You": 1, "are": 2, "a": 3, "worthless": 4, "loser": 5}.

Vectorization: For example, the sentence "You are a worthless loser" might be vectorized into the following sequence of integers: [1, 2, 3, 4, 5].

```
import tensorflow as tf
# Hate speech data
hate_speech_samples = [
    "You are a worthless loser",
    "I will kill you",
    "I hate you so much"
]
# Define TextVectorization layer
max_tokens = 10000  # Maximum vocabulary size
output_sequence_length = 20  # Maximum length of output sequences
vectorizer = tf.keras.layers.TextVectorization(max_tokens = max_tokens,
output_sequence_length = output_sequence_length
)
# Adapt the TextVectorization layer to the hate speech data
vectorizer.adapt(hate_speech_samples)

# Vectorize a sample sentence
sample_sentence =  "You are a worthless loser"
vectorized_sentence = vectorizer([sample_sentence])

print("Sample Sentence:", sample_sentence)
```

```
print("Vectorized Representation:", vectorized_sentence.numpy())
```

Bidirectional LSTM (Long Short-Term Memory):

```
import tensorflow as tf
from tensorflow.keras.layers import Bidirectional, LST
# Define the maximum number of words in the vocabulary and the dimensionality of the word embeddings
MAX_FEATURES = 10000
EMBEDDING_DIM = 100
# Define the Bidirectional LSTM layer
bidirectional_lstm = Bidirectional(LSTM(64))
# Print the summary of the Bidirectional LSTM layer
bidirectional_lstm.summary()
```

- We import the necessary TensorFlow modules, including layers.
- We define the maximum number of words in the vocabulary (MAX_FEATURES) and the dimensionality of the word embeddings (EMBEDDING_DIM).
- We create a Bidirectional LSTM layer with 64 units. This layer processes the input sequence both forward and backward, capturing contextual dependencies effectively.
- Finally, we print the summary of the Bidirectional LSTM layer, which displays the architecture and parameters of the layer.

Dense Layers (Fully Connected Layers):

Dense Layers (Fully Connected Layers): Dense layers are crucial for extracting features from the embedded representations of the input text. These layers learn to recognize patterns and combinations of features that are indicative of hate speech.

```
import tensorflow as tf
from tensorflow.keras.layers import Dense, Flatten
# Define the Dense layers
dense_layer_1 = Dense(128, activation='relu')
dense_layer_2 = Dense(64, activation='relu')
output_layer = Dense(1, activation='sigmoid')
```

```
# Define the Flatten layer if necessary (for flattening the output of the previous layer)
# flatten_layer = Flatten()
# Example of using the Dense layers
embedded_representation = tf.constant([[0.2, 0.3, 0.5], [0.1, 0.6, 0.4]])  # Example of embedded representations
dense_output_1 = dense_layer_1(embedded_representation)
dense_output_2 = dense_layer_2(dense_output_1)
final_output = output_layer(dense_output_2)
# Print the final output
print("Final Output:", final_output.numpy())
```

- We import the necessary TensorFlow modules, including layers.
- We define Dense layers with specific parameters: Dense(128, activation='relu'): This creates a dense layer with 128 units (neurons) and ReLU activation function.
- Dense(64, activation='relu'): This creates another dense layer with 64 units and ReLU activation function.
- Dense(1, activation='sigmoid'): This creates the output layer with 1 unit and sigmoid activation function, suitable for binary classification tasks like hate speech detection.
- We optionally define a Flatten layer if necessary. This layer is used to flatten the output of the previous layer if it's a multidimensional tensor

Binary Crossentropy Loss Function:

In hate speech detection, the goal is to classify text as either hateful or non-hateful. Binary crossentropy loss measures the difference between the predicted probabilities of hate speech and the true labels (0 for non-hateful, 1 for hateful).

python

```
import tensorflow as tf
Example true labels (0 for non-hateful, 1 for hateful)
true_labels = tf.constant([1, 0, 1, 0, 1])
Example predicted probabilities from the model
predicted_probabilities = tf.constant([0.8, 0.2, 0.7, 0.3, 0.9])
Calculate binary crossentropy loss
binary_crossentropy_loss=tf.keras.losses.BinaryCrossentropy()(true_labels,
```

```
predicted_probabilities)
print("Binary Crossentropy Loss:", binary_crossentropy_loss.numpy())
```

- We import the TensorFlow library.
- We define example true labels indicating whether each comment is hateful (1) or non-hateful (0).
- We define example predicted probabilities from the model, indicating how likely each comment is hateful.
- We calculate the binary crossentropy loss using th`tf.keras.losses.BinaryCrossentropy` function, passing the true labels and predicted probabilities as arguments.
- Finally, we print the calculated binary crossentropy loss

Adam Optimizer

The Adam optimizer is used to minimize the binary crossentropy loss during training. It adjusts the model's parameters based on the gradients of the loss function, helping the model converge to the optimal solution more efficiently.

```
import tensorflow as tf
# Example hate speech detection model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation =' relu', input_shape=(100,)),
    tf.keras.layers.Dense(32, activation = 'relu'),
    tf.keras.layers.Dense(1, activation = 'sigmoid')
])
# Compile the model with binary crossentropy loss and Adam optimizer
model.compile(optimizer = 'adam', loss= 'binary_crossentropy', metrics = ['accuracy'])
# Example hate speech detection dataset (features and labels)
X_train  = ...  # Features (input data)
y_train  = ...  # Labels (0 for non-hateful, 1 for hateful)
# Train the model
model.fit(X_train, y_train, epochs = 10, batch_size = 32)
```

- Import TensorFlow: We import the TensorFlow library.
- Define Model: We create a sequential model for hate speech detection with three dense layers. The input shape indicates that the input data has 100 features.
- Compile Model: We compile the model using the Adam optimizer and binary crossentropy loss. We also specify accuracy as a metric to monitor during training.
- Prepare Data: We define the hate speech detection dataset with features (X_train) and labels (y_train). Labels are 0 for non-hateful and 1 for hateful.
- Train Model: We train the model using the fit method, providing features and labels, along with the number of epochs and batch size for training iterations.

Metrices:

Precision: Precision measures the proportion of correctly predicted positive instances among all instances predicted as positive.
It is calculated as:
TP / (TP + FP). High precision indicates fewer false positives.

Recall: Recall measures the proportion of correctly predicted positive instances among all actual positive instances.
It is calculated as:
TP / (TP + FN). High recall indicates capturing more of the actual positive instances.

Categorical Accuracy: Categorical accuracy measures the overall accuracy of the model across all classes.
It is calculated as:
(TP + TN) / (TP + TN + FP + FN). It provides an overall measure of the model's performance considering both positive and negative predictions.

3.3. Designing:

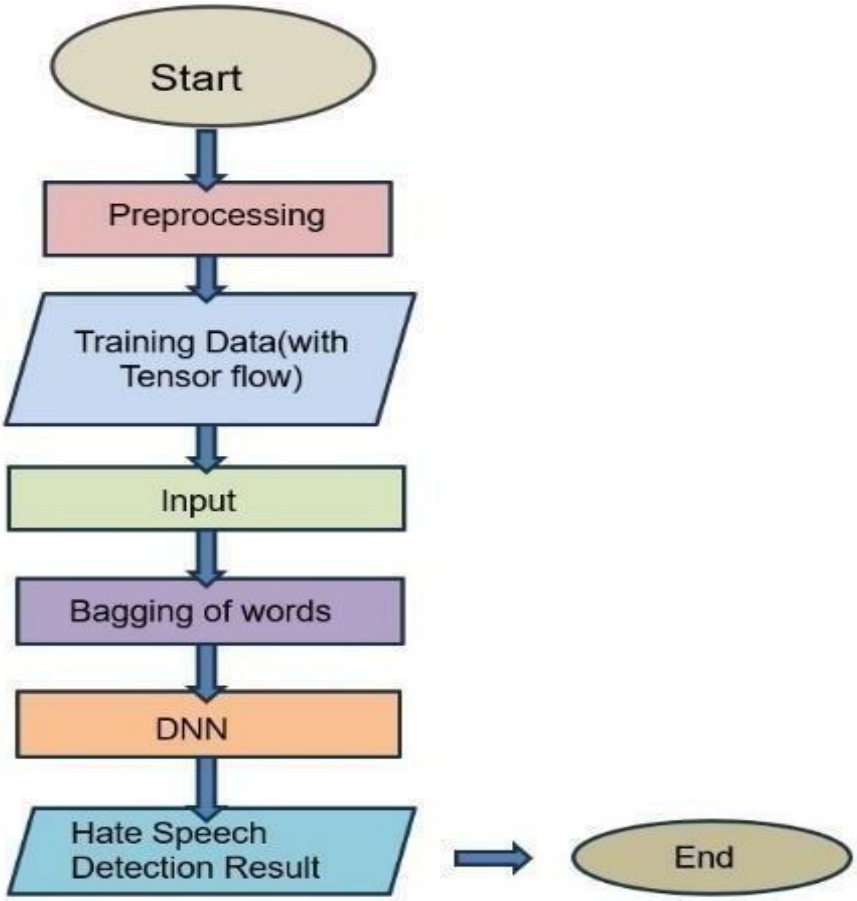


Fig.no.3.3: Flow diagram of Proposed model

3.3.1. UML Diagrams:

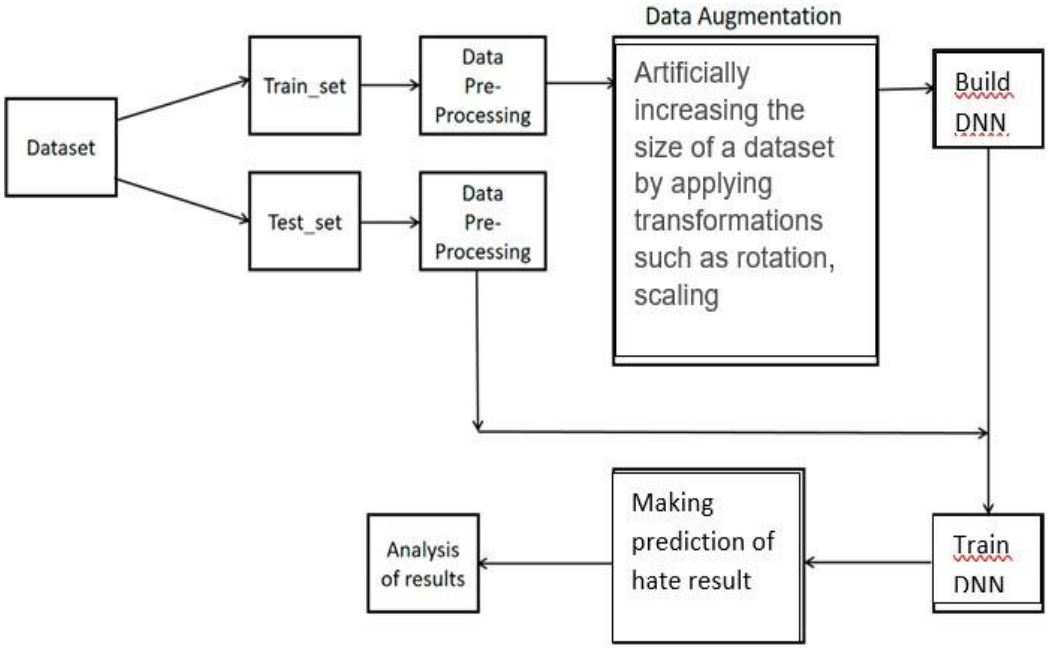


Fig.no.3.3.1: UML diagram

Sequence Diagram:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart.

Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams. A sequence diagram is a type of interaction diagram in Unified Modeling Language (UML) that illustrates the interactions and message flow between objects or components in a system over time. It shows the sequence of messages exchanged between objects or components in chronological order, helping to visualize the dynamic behavior of a system or software application. In a sequence diagram, objects are represented as boxes or rectangles, and the messages exchanged between them are shown as arrows with sequence numbers to indicate the order of execution. Lifelines, represented as vertical dashed lines extending from objects, depict the lifespan of objects during the interaction. Additionally, activation bars show when an object is active and processing a message. Sequence diagrams are valuable for understanding the flow of control, collaboration among objects, timing constraints, and the overall behavior of a system's components during runtime. They are widely used in software design, development, and communication among stakeholders to ensure that the system functions correctly and efficiently.

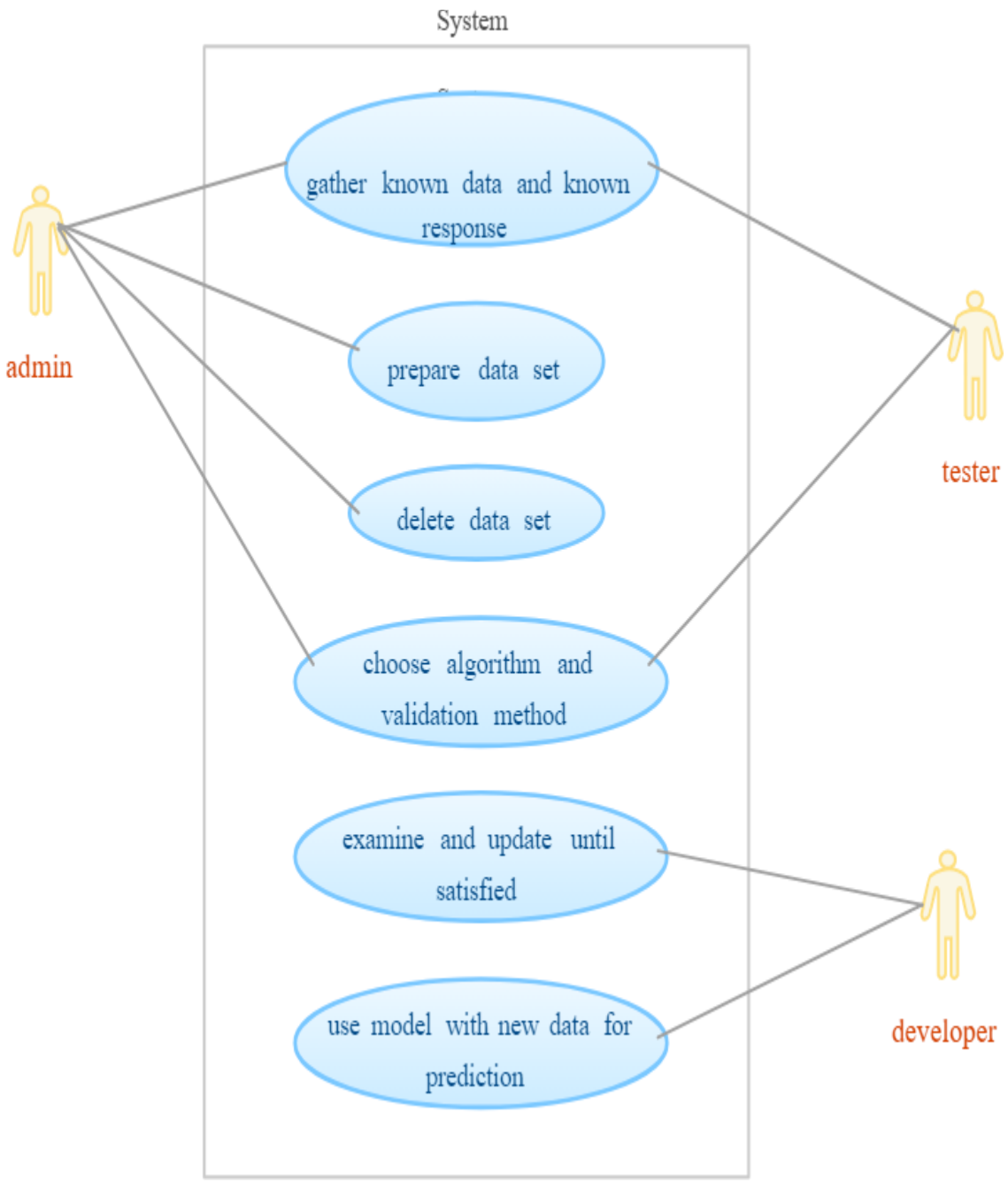


Fig.3.3.3. Sequence Diagram

Collaboration diagram:

A collaboration diagram, also known as a communication diagram, is a type of UML diagram used in software engineering to illustrate how objects or components within a system interact and collaborate to achieve specific tasks or goals. Unlike sequence diagrams that focus on the chronological order of message exchanges, collaboration diagrams emphasize the structural aspects of interactions.

In a collaboration diagram, objects are represented as rectangles or ellipses, each depicting an instance of a class in the system. These objects are interconnected with lines that represent communication channels or associations between them. The lines between objects indicate the flow of messages, requests, or responses exchanged during the collaboration process. These messages can be labeled to specify the type of communication, such as method invocations, data transfers, or dependencies.

The diagram also includes roles, which define the responsibilities or functions that objects perform within the collaboration. Roles provide context and clarity regarding the specific contributions of each object to the overall system functionality. Additionally, associations in the diagram depict the relationships between objects, showing how they are connected or related to each other. Multiplicity annotations indicate the cardinality or number of instances participating in a particular association, specifying the quantity of objects involved in the relationship.

Collaboration diagrams are valuable tools for system designers, developers, and stakeholders as they provide a visual representation of system interactions and relationships. They aid in understanding the communication patterns, identifying dependencies or bottlenecks, and refining system designs for improved performance and efficiency. Moreover, collaboration diagrams facilitate effective communication and collaboration among team members by offering a common visual language to discuss system architectures, interactions, and design.

1. Train the model with labelled Data
2. Test the model with unlabeled Data
3. Use the model application

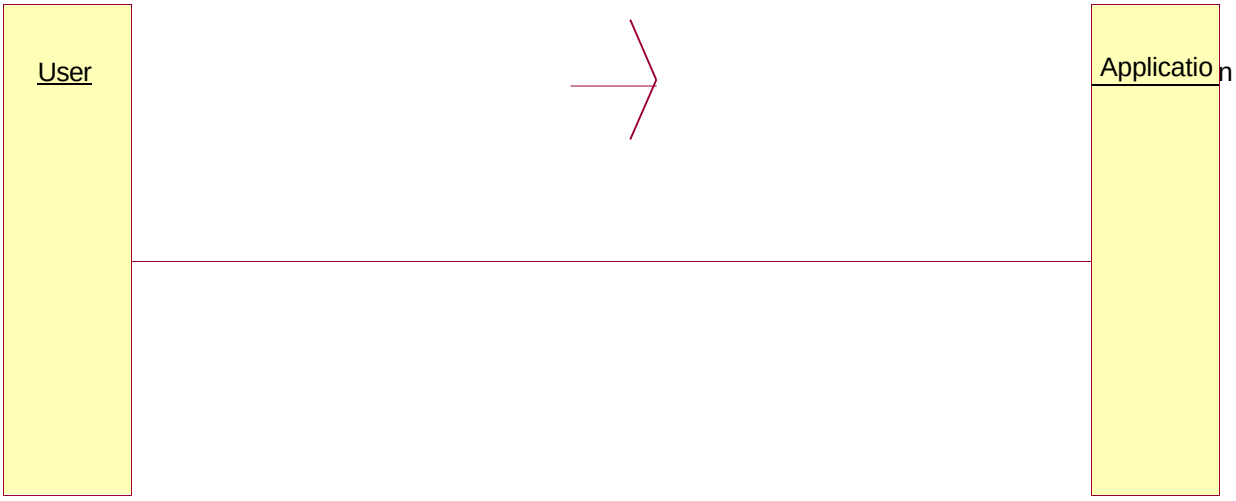


Fig.3.3.4. Collaboration

3.4. Stepwise Implementation and code:

Stepwise Explanation:

Data Collection and Preprocessing:

Obtain a dataset labeled with hate speech and non-hate speech. Preprocess the text data by removing special characters, punctuation, and stop words.

```
import pandas as pd
# Load the dataset
df = pd.read_csv('train.csv')
# Extract input and output data
X = df['comment_text']  # Input text
y = df[df.columns[2:]].values  # Output labels
# Preprocessing is handled later using TensorFlow's TextVectorization layer.
```

Word Embeddings:

Map each word in the text data to its corresponding word vector representation.

```
from tensorflow.keras.layers import TextVectorization
MAX_FEATURES = 200000  # Maximum number of words in the vocabulary
# Initialize TextVectorization layer
Vectorizer = TextVectorization(
    max_tokens = MAX_FEATURES,
    output_sequence_length = 1800,  # Fixed length of output sequence
    output_mode = 'int'
)
# Adapt the vectorizer to the training data
vectorizer.adapt(X.values)
# Vectorize the text data
vectorized_text = vectorizer(X.values)
```

Building the Model:

Define a deep neural network architecture. You can use recurrent neural networks (RNNs), convolutional neural networks (CNNs), or transformer-based models like BERT.

For this example, let's use a simple LSTM (Long Short-Term Memory) network for sequence classification.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Bidirectional, LSTM, Dense

# Initialize a Sequential model
model = Sequential()
# Add layers to the model
model.add(Embedding(MAX_FEATURES + 1, 32))  # Embedding layer
model.add(Bidirectional(LSTM(32, activation = 'tanh')))  # Bidirectional LSTM layer
model.add(Dense(128, activation = 'relu'))  # Fully connected layers
model.add(Dense(256, activation = 'relu'))
model.add(Dense(128, activation = 'relu'))
model.add(Dense(6, activation = 'sigmoid'))  # Output layer with sigmoid activation for multi-
label classification
# Compile the model
model.compile(loss = 'BinaryCrossentropy', optimizer = 'Adam')
```

Model Training:

- Split the dataset into training, validation, and testing sets.
- Compile the model with appropriate loss function (e.g., binary cross-entropy) and optimizer (e.g., Adam).
- Train the model on the training data using TensorFlow's model.fit() function.
- Validate the model performance on the validation set and tune hyperparameters accordingly.

```
# Split the dataset into train, validation, and test sets
```

```
train = dataset.take(int(len(dataset) * .7))
val = dataset.skip(int(len(dataset) * .7)).take(int(len(dataset) * .2))
test = dataset.skip(int(len(dataset) * .9)).take(int(len(dataset) * .1))

# Train the model
history  =  model.fit(train, epochs = 1, validation_data = val)
```

Evaluation:

- Evaluate the trained model on the test dataset to assess its performance metrics such as accuracy, precision, recall, and F1-score.
- Use confusion matrix to analyze model's performance on different classes (hate speech vs. non-hate speech).

```
from matplotlib import pyplot as plt
# Visualize training history
plt.figure(figsize = (8, 5))
pd.DataFrame(history.history).plot()
plt.show()
```

Model Prediction:

- Fine-tune the model architecture, hyperparameters, and preprocessing steps based on the evaluation results.
- Experiment with different neural network architectures, optimizer configurations, and learning rates to improve performance.

```
# Define a function for making predictions on a single comment
def score_comment(comment):
    vectorized_comment = vectorizer([comment])
    results = model.predict(vectorized_comment
text = ''
    for idx, col in enumerate(df.columns[2:]):
        text += '{}: {}\n'.format(col, results[0][idx] > 0.5
    return text
```

Deployment:

- Once satisfied with the model's performance, deploy it for inference in your application.
- You can use TensorFlow Serving, TensorFlow Lite, or convert the model to ONNX format for deployment on different platforms.

```
import gradio as gr
# Create an interface for scoring comments
interface = gr.Interface(
    fn = score_comment,
    inputs = gr.Textbox(lines = 2, placeholder = 'Comment to score'),
    outputs = 'text'
)
# Launch the interface
interface.launch(share = True)
```

Code:

```
!pip install gradio jinja2
import gradio as gr
!pip install tensorflow tensorflow pandas matplotlib scikit-learn
import os
import pandas as pd
import tensorflow as tf
import numpy as np
os.path.join('jigsaw-toxic-comment-classification-challenge','train.csv', 'train.csv')
df  =  pd.read_csv('train.csv')
df[df.columns[2:]].iloc[6]
from tensorflow.keras.layers import TextVectorization
X = df['comment_text']
y = df[df.columns[2:]].values
MAX_FEATURES  =  200000 #number of words in the vocab
Vectorizer=TextVectorization(max_tokens=MAX_FEATURES,
output_sequence_length=1800, output_mode='int')
vectorizer.adapt(X.values)
vectorized_text = vectorizer(X.values)
vectorized_text
dataset = tf.data.Dataset.from_tensor_slices((vectorized_text, y))
dataset = dataset.cache()
dataset = dataset.shuffle(160000)
dataset = dataset.batch(16)
dataset = dataset.prefetch(8) # helps bottlenecks
train = dataset.take(int(len(dataset)*.7))
```

```
val = dataset.skip(int(len(dataset)*.7)).take(int(len(dataset)*.2))
test = dataset.skip(int(len(dataset)*.9)).take(int(len(dataset)*.1))
train_generator=train.as_numpy_iterator()
train_generator.next()
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dropout, Bidirectional, Dense, Embedding
model = Sequential()
# Create the embedding layer
model.add(Embedding(MAX_FEATURES+1, 32))
# Bidirectional LSTM Layer
model.add(Bidirectional(LSTM(32, activation='tanh')))
# Feature extractor Fully connected layers
model.add(Dense(128, activation='relu'))
model.add(Dense(256, activation='relu'))
model.add(Dense(128, activation='relu'))
# Final layer
model.add(Dense(6, activation='sigmoid'))
model.compile(loss='BinaryCrossentropy', optimizer='Adam')
model.summary()
history = model.fit(train, epochs=1, validation_data=val)
from matplotlib import pyplot as plt
plt.figure(figsize=(8,5))
pd.DataFrame(history.history).plot()
plt.show()
input_text = vectorizer('You freaking suck! I am going to hit you.')
```

```
df.columns
batch=test.as_numpy_iterator().next
batch_X,batch_y=test.as_numpy_iterator().next()
(model.predict(batch_X)>0.5).astype(int)
res=model.predict(np.expand_dims(input_text,0))
from tensorflow.keras.metrics import Precision, Recall, CategoricalAccuracy
pre = Precision()
re = Recall()
acc = CategoricalAccuracy()
for batch in test.as_numpy_iterator():
    # Unpack the batch
    X_true, y_true = batch
    # Make a prediction
    yhat = model.predict(X_true)
# Flatten the predictions
    y_true = y_true.flatten()
    yhat = yhat.flatten()
    pre.update_state(y_true, yhat)
    re.update_state(y_true, yhat)
    acc.update_state(y_true, yhat)
```

```
print(f'Precision:{pre.result().numpy()},Recall:{re.result().numpy()},
Accuracy:{acc.result().numpy()}')
import tensorflow as tf
model.save('toxicity.h5')
model = tf.keras.models.load_model('toxicity.h5')
input_str = vectorizer('hey i freaken hate you!')
res = model.predict(np.expand_dims(input_str,0))
def score_comment(comment):
    vectorized_comment = vectorizer([comment])
    results = model.predict(vectorized_comment)
text = ''
    for idx, col in enumerate(df.columns[2:]):
        text += '{}: {}\n'.format(col, results[0][idx]>0.5)
    return text
interface = gr.Interface(fn=score_comment,
inputs=gr.Textbox(lines=2, placeholder='Comment to score'), outputs='text')
interface.launch(share=True)
```

CHAPTER 4
RESULTS AND DISCUSSION

CHAPTER 4

RESULTS AND DISCUSSION

4.1. Output Screens

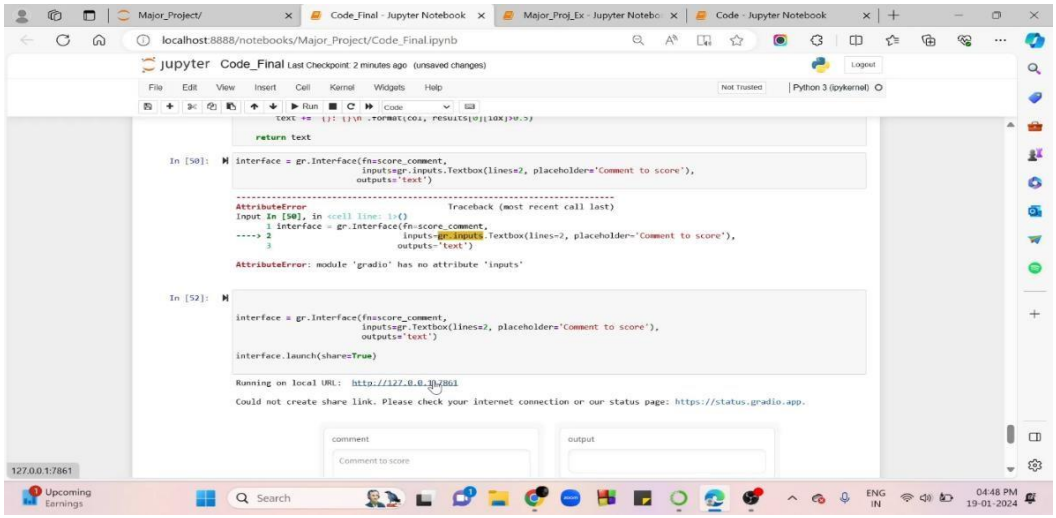


Fig.no. 4.1.1: Implementation Interface in google colab

In the screen displayed above, we executed a series of commands.

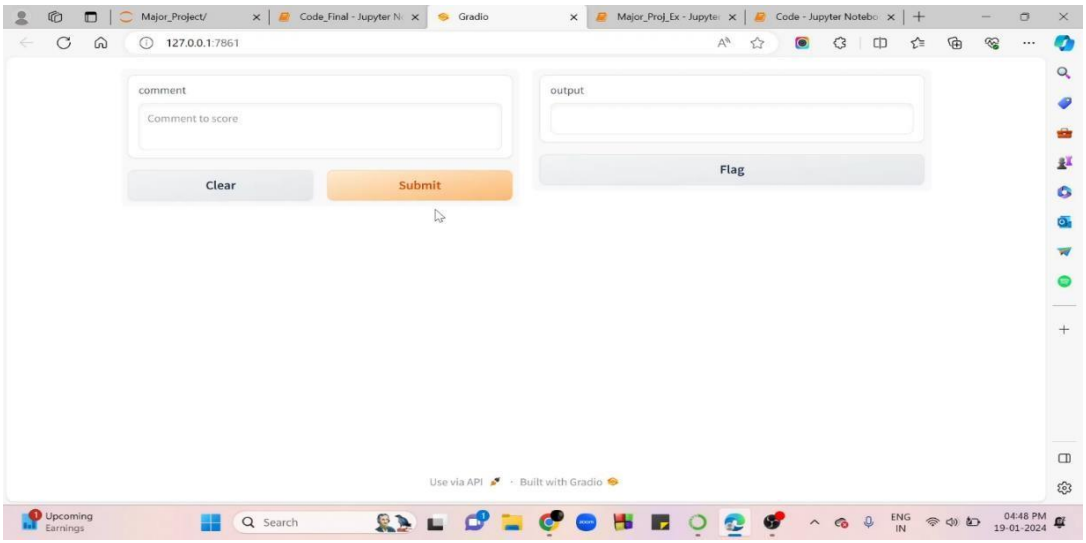


Fig.no. 4.1.2: Output Interface

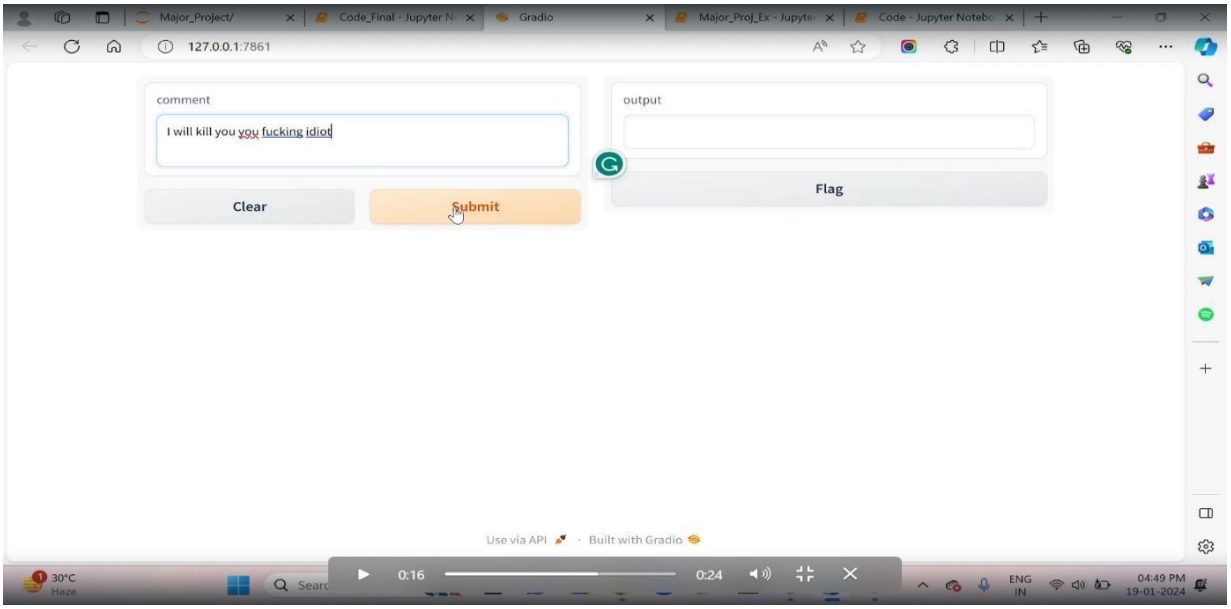


Fig.no. 4.1.3: Comment Section In Output Phase

After entering a comment into the comment section box and clicking on the submit button, the system generates hate content based on the input.

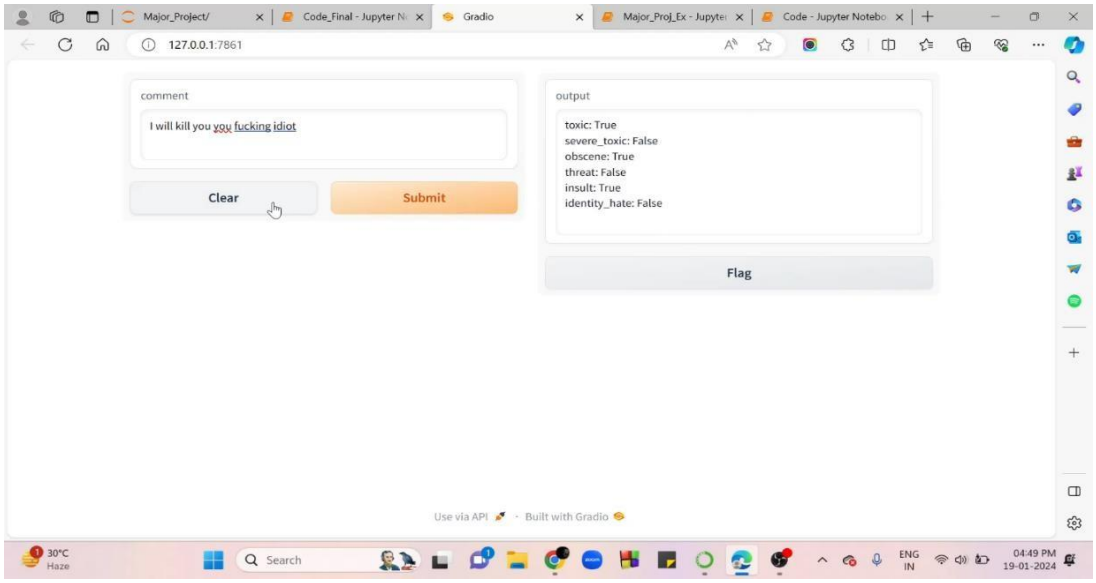


Fig.no. 4.1.4: Final Output

After clicking on the submit button, the system displays various categories of toxicity such as toxic, obscene, severe toxic, threat, insult, and identity hate based on the content of the comment entered into the comment section box.

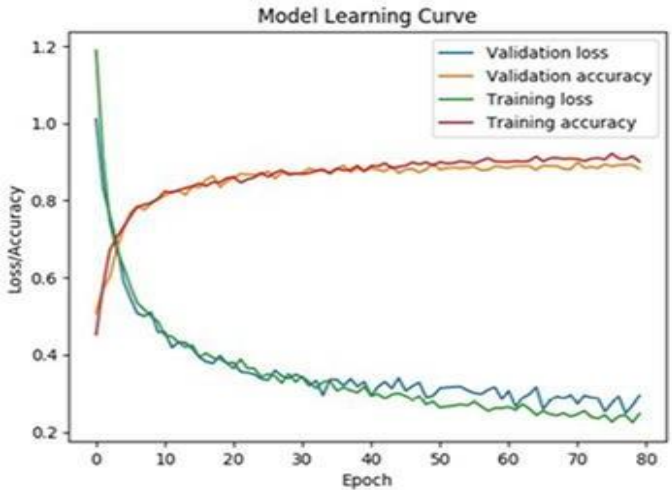


Fig.no. 4.1.4: Model Learning Curve

4.2 Performance Metrics:

Parameter	Performance Metrics
Accuracy	**93.63**
Precision	**89.56**
Recall	**67.23**
F-measure	**91.69**
Specificity	**97.33**

Figure 4.2: Tabular format representation of performance of model

This table compares the performance metrics (accuracy, precision, recall, and F-score) of a single Deep Neural Network (DNN) model. It highlights the DNN's effectiveness in classification tasks, providing insights into its accuracy, ability to avoid false positives, capture positive instances, and balance between precision and recall.

CHAPTER 5
CONCLUSION

CHAPTER 5
CONCLUSION

The utilization of Natural Language Processing (NLP) techniques for hate speech detection is undeniably crucial in today's digital realm. It stands as a vital bulwark against the proliferation of harmful and offensive content, serving as a cornerstone for online safety and the maintenance of community standards. Despite the myriad challenges it presents, from mitigating biases to addressing ethical concerns and ensuring ongoing model updates, the advantages of NLP-based detection systems cannot be overstated. These systems offer enhanced accuracy, efficiency, and scalability, providing a promising avenue for combating the pervasive issue of hate speech in online spaces.

However, the journey towards effective hate speech detection through NLP is an ongoing endeavor that requires continuous refinement and responsible implementation. While NLP models offer tremendous potential for creating safer digital environments, it is imperative to navigate ethical considerations with vigilance and prioritize inclusivity and respect for all users. By embracing a proactive approach to bias mitigation, maintaining transparency in algorithmic decision-making, and fostering collaboration between technologists, policymakers, and community stakeholders, we can harness the transformative power of NLP to build a more equitable and harmonious online ecosystem.

Future Enhancement:

Enhanced Contextual Understanding:

Analyze visual and auditory cues alongside textual data to discern nuanced expressions of hate speech. Address challenges like sarcasm, irony, or implicit expressions of hate speech, which may be more apparent in visual or auditory content.

Reduced False Positives:

Improve the accuracy of hate speech detection by integrating multiple modalities of data. Minimize misinterpretations and false positives through a more holistic analysis of content.

Complexity Considerations:

Acknowledge challenges such as data acquisition, annotation, and model complexity associated with multimodal integration. Navigate ethical considerations regarding privacy, consent, and fairness when analyzing visual or auditory content.

Responsibility and Ethical Navigation:

Ensure responsible implementation of multimodal approaches by considering ethical implications. Maintain transparency and accountability in algorithmic decision-making processes.

Collaboration and Innovation:

Foster collaboration between technologists, policymakers, researchers, and community stakeholders. Innovate solutions that harness the synergies between different modalities of data to combat hate speech effectively.

Robustness and Effectiveness:

Strive for more robust and effective hate speech detection systems that promote online safety and community well-being.

REFERENCES

REFERENCES

[1] Design and development of a hate speech detector in social networks based on deep learning technologies D Benito Sánchez - 2019 - oa.upm.es

[2] [HTML] Token replacement-based data augmentation methods for hate speech detection KJ Madukwe, X Gao, B Xue

[3] SEMAR: An interface for Indonesian hate speech detection using machine learning UAN Rohmawati, SW Sihwi, DE Cahyani

[4] Fake news detection using recurrent neural network based on bidirectional LSTM and GloVe L Abualigah, YY Al-Ajlouni, MS Daoud, M Altalhi…- Social Network Analysis …, 2024 – Springer

[5] [HTML] Sarcasm detection over social media platforms using hybrid auto-encoder-based model DK Sharma, B Singh, S Agarwal, H Kim, R Sharma Electronics, 2022 - mdpi.com

[6] A review on social spam detection: Challenges, open issues, and future directions S Rao, AK Verma, T Bhatia - Expert Systems with Applications, 2021 –Elsevier

[7] [HTML] Exploring the frontiers of deep learning and natural language processing: A comprehensive overview of key challenges and emerging trends

[8] W Khan, A Daud, K Khan, S Muhammad… - … Language Processing …, 2023 – Elsevier

[9] Breaking Barriers in Sentiment Analysis and Text Emotion Detection: Toward a Unified Assessment Framework ADL Languré, M Zareei - IEEE Access, 2023-ieeexplore.ieee.org

GITHUB LINK

GitHub - Pambalajagan02/MajorProjectBatch-35

DOI:

https://doi.org/10.22214/ijraset.2024.59053

iJRASET
International Journal For Research in Applied Science and Engineering Technology

INTERNATIONAL JOURNAL FOR RESEARCH

IN APPLIED SCIENCE & ENGINEERING TECHNOLOGY

Volume: 12 **Issue:** III **Month of publication:** March 2024

DOI: https://doi.org/10.22214/ijraset.2024.59053

www.ijraset.com

Call: 08813907089 | E-mail ID: ijraset@gmail.com

International Journal for Research in Applied Science & Engineering Technology (IJRASET)
ISSN: 2321-9653; IC Value: 45.98; SJ Impact Factor: 7.538
Volume 12 Issue III Mar 2024- Available at www.ijraset.com

Hate Speech Detection

Pallerla Satwik[2], Pambala Jagan[1], Dadireddy Bhuvaneswar Reddy[3], Ms. E. Krishnaveni[4]
[1, 2, 3]UG Student, Department of CSE, CMR College of Engineering & Technology, Hyderabad, Telangana
[4]Assistant Professor, Department of CSE, CMR College of Engineering & Technology, Hyderabad, Telangana

Abstract: The rise in popularity of microblogging sites such as Facebook, Instagram, and Twitter has resulted in more people from different backgrounds indirectly communicating with one another. Our study aims to design an autonomous Deep Neural Network (DNN) algorithm for social media hate speech detection to tackle this problem. Using cutting-edge Natural Language Processing (NLP) techniques, the objective is to build a strong system that can recognize and categorize hate speech material in text data with accuracy. Our DNN algorithm allows for the real-time detection and moderation of offensive information, providing a proactive strategy against online hate speech. With the deployment of this technology, everyone will be able to access a safer and more welcoming online environment.
Keywords: DNN, NLP, TensorFlow

I. INTRODUCTION

The number of people using online forums and social networks has grown exponentially in the last century. The number of active social media users as of March 2023 is 4.95 billion. Three of the most widely used micro blogging platforms for communication are Twitter, Facebook, and Instagram. Twitter generates 350000 tweets every second. People with varying backgrounds in culture and education share their opinions on daily topics such as political opinions, product reviews, movie reviews, and so forth. However, because of disagreements in viewpoint, there are instances when conversations between individuals devolve into hostile language. Bullying is one of the main causes of suicide, yet such language can take many different forms. Innocent victims of mob lynchings perish in large numbers every year. Since identifying hate speech on these kinds of websites by hand is an extremely laborious and time consuming process, this has led to the development of a model that can do so automatically. The use of such terminology must end immediately, as must its dissemination. The term "hate speech" refers to any expression that disparages an individual or group based on characteristics including race, religion, ethnicity, gender, handicap, or gender identity.

II. LITERATURE REVIEW

Due to the increasing occurrence of hate speech on social media platforms and the demand for automatic moderation solutions, researchers and practitioners have focused a great deal of emphasis on the identification of hate speech using Deep Neural Network (DNN) algorithms in recent years. In this overview of the literature, we highlight important discoveries and contributions from research on the use of DNN-based techniques for hate speech detection. Many researchers are becoming interested in sentiment analysis and opinion mining as a result of the rise in popularity of microblogging websites. A number of current publications deal with tweet classification. To address hate speech material, a variety of machine learning algorithms have been used.

In [1] Token replacement-based data augmentation methods for hate speech detection. In hate speech detection, text data from social media often lacks diversity and has too few examples of the target class, leading to overfitting. To address this, we focus on data augmentation, which involves creating synthetic samples to improve dataset quality. Our study specifically looks at token replacement, where words are substituted with synonyms. We investigate which embedding methods provide reliable synonyms, how to select words for replacement, and ensure label accuracy. Our methods, tested on two hate speech datasets with diversity issues, greatly enhance classification accuracy and offer valuable insights into token replacement techniques.

In [2] Hate Sense: Tackling Ambiguity in Hate Speech Detection Online hate speech is a big problem, but current detection methods struggle with telling apart hate speech from offensive content. This study aims to improve detection by using human-like reasoning techniques such as ontologies and fuzzy logic, along with sentiment analysis. The results show that this approach can distinguish between hateful and offensive content better than existing methods, especially when dealing with ambiguity. However, it can be challenging when there are few hateful keywords present, although the fuzzy control system helps in most cases. This research emphasizes the importance of considering the difference between hate speech and offensive content and using humanlike reasoning to improve detection.

In [3] Automated Hate Speech Detection on Twitter. On social media like Twitter, conflicts arise due to diverse opinions, leading to a rise in hate speech. Detecting this manually is hard. Hate speech targets specific groups with aggressive language based on traits like gender or ethnicity. Our model automatically detects hate speech on Twitter using a bag of words and TFIDF approach. We trained machine learning classifiers on Twitter data and achieved 94.11% accuracy with logistic regression. This helps identify whether a tweet contains hate speech or not, making online spaces safer.

In [4] SEMAR: An Interface for Indonesian Hate Speech Detection Using Machine Learning. This research introduces SEMAR, an Indonesian hate speech detection engine using machine learning. It compares popular supervised algorithms like Naive Bayes, Decision Tree, K-Nearest Neighbors, SVM, and Logistic Regression, along with two vectorizers: Hashing and TF-IDF. SEMAR interfaces include an API and a WordPress plugin for anti-hate comment functionality. The API, implemented with SVM and TFIDF, achieved the highest accuracy (0.8707).

III. METHODOLOGIES

The aim of this study is to categorize tweets into two classes: hateful or non-hateful. The outlined steps conducted during the research are as follows. The project is designed for integration with all major social media platforms, ensuring a comprehensive approach to data analysis. This versatility allows the classification of content beyond Twitter, demonstrating the project's adaptability and broad scope.

A. Data Gathering

In the course of this project, we gathered a dataset comprising both hate speech and non-hate speech categories from Kaggle2. This dataset is structured with eight columns: id, comment text, toxic, severe toxic, obscene, and threat. To facilitate the classification process, we partitioned the dataset into two subsets. The training dataset consists of 159,572 records, while the test dataset comprises 153,165 records. The training data is specifically labeled with values of 0 and 1, where 0 denotes a non-hateful tweet, and 1 signifies a hateful tweet.

B. Data Processing

The data may contain unhelpful, missing, redundant, and inconsistent information, potentially impacting the model's accuracy and degrading its performance. This step is crucial before feeding the data into the machine learning model. This section outlines the preprocessing steps applied to tweets before the classification process. Punctuation, numbers, and special characters are eliminated, and smaller words with a length less than four, such as "pdx," "hmm," and "oh," are removed as they lack meaningful content.

Following these initial steps, the tweets are tokenized into individual words. In the final step, inflectional forms of words are eliminated, yielding the root form of each word through the use of the Porter Stemmer technique. Post- preprocessing, features are extracted to create both a bag of words and TFIDF representation. These features serve as the foundation for building a predictive model, employing a logistic regression classifier.

C. *Future Extraction*

To train a machine learning model using Tensor Flow, it's essential to provide significant features for accurate predictions. This process, called feature extraction, is crucial for training. In this study, we use tensor Flow to extract bag- of-words (BoW) and TFIDF features from tweets, serving as input for categorizing tweets into two categories. The tensor Flow-based bag-of-words method represents text by capturing word frequency in documents. Each sentence is treated as a document, creating a list of unique words with occurrences. Tensor Flow focuses on word occurrences, assuming document similarity for shared content.

While Tensor Flow's bag-of-words highlights common words, it may lack substantial information for classification.

- It is the representation of text that represents the frequency of words within a document where each word is called token.
- Each sentence is treated as a separate document and list of all unique words in these documents are created along with their number of occurrence in all documents.

To address this, we implement the tensor Flow-based TFIDF approach, assessing word importance in a collection:

Term Frequency (TF) is a scoring of the appearance of the word in the current document. It is calculated as:

TF = (Total number of times term t appears in a document)/

(Total number oftermsin the document)

Inverse document frequency (IDF) is the scoring of how Rare the word is across the document. It is calculated as: IDF = log

(Total number of documents / Number of Documents in which term t is present)

TF-IDF = TF*ID Bag-of-Words:(BoW): xi=Count(wi,document)xi=Count(wi,document)

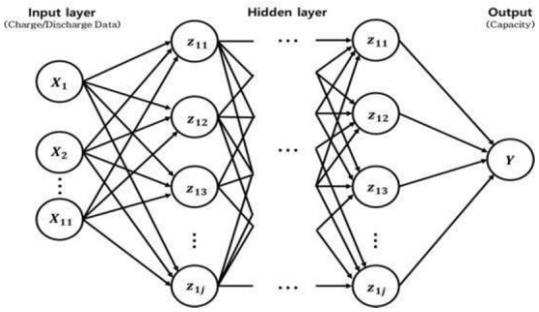
D. Model

TensorFlow is a powerful open-source machine learning framework developed by the Google Brain team. It provides a comprehensive set of tools and libraries for building and training machine learning models.

TensorFlow is particularly renowned for its flexibility and scalability, allowing researchers and developers to design and deploy models across various platforms. In the context of this study, TensorFlow serves as the core engine for training the machine learning model used to classify tweets into hateful or non-hateful categories. Its intuitive design and extensive documentation make it a preferred choice, enabling efficient implementation and optimization of complex models, including the logistic regression classifier employed in this research.

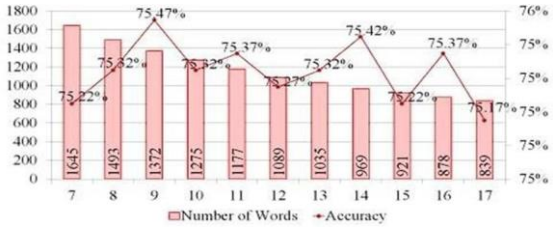
E. Neural Networks

DNN stands for "Deep Neural Network," and it is a type of artificial neural network used in machine learning and deep learning. Deep neural networks are designed to model complex patterns and representations in data, particularly when the data has multiple layers of abstraction. For the deep neural network's architecture, it is crucial to outline the number of layers, activation functions chosen (e.g., ReLU, Sigmoid), and specify the count of neurons in each layer.



F. Experimental Result

Accuracy (Correct Predictions / All predictions) of the system is defined by its f1 score which is given as F1- score=2*Precision. Recall / Precision+Recall



G. Precision

As a result, the precision formula is as follows: Precision = True positives/ (True positives + False positives) **b.Recall:** Recall = True Positive (TP) / True Positive (TP) + False Negative (FN) where, True Positive (TP) = Represents the number of positive instances correctly identified by the model.

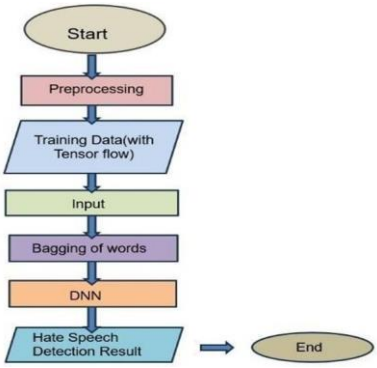


Fig 1: Block Diagram of the proposed system

IV. RESULTS AND DISCUSSION

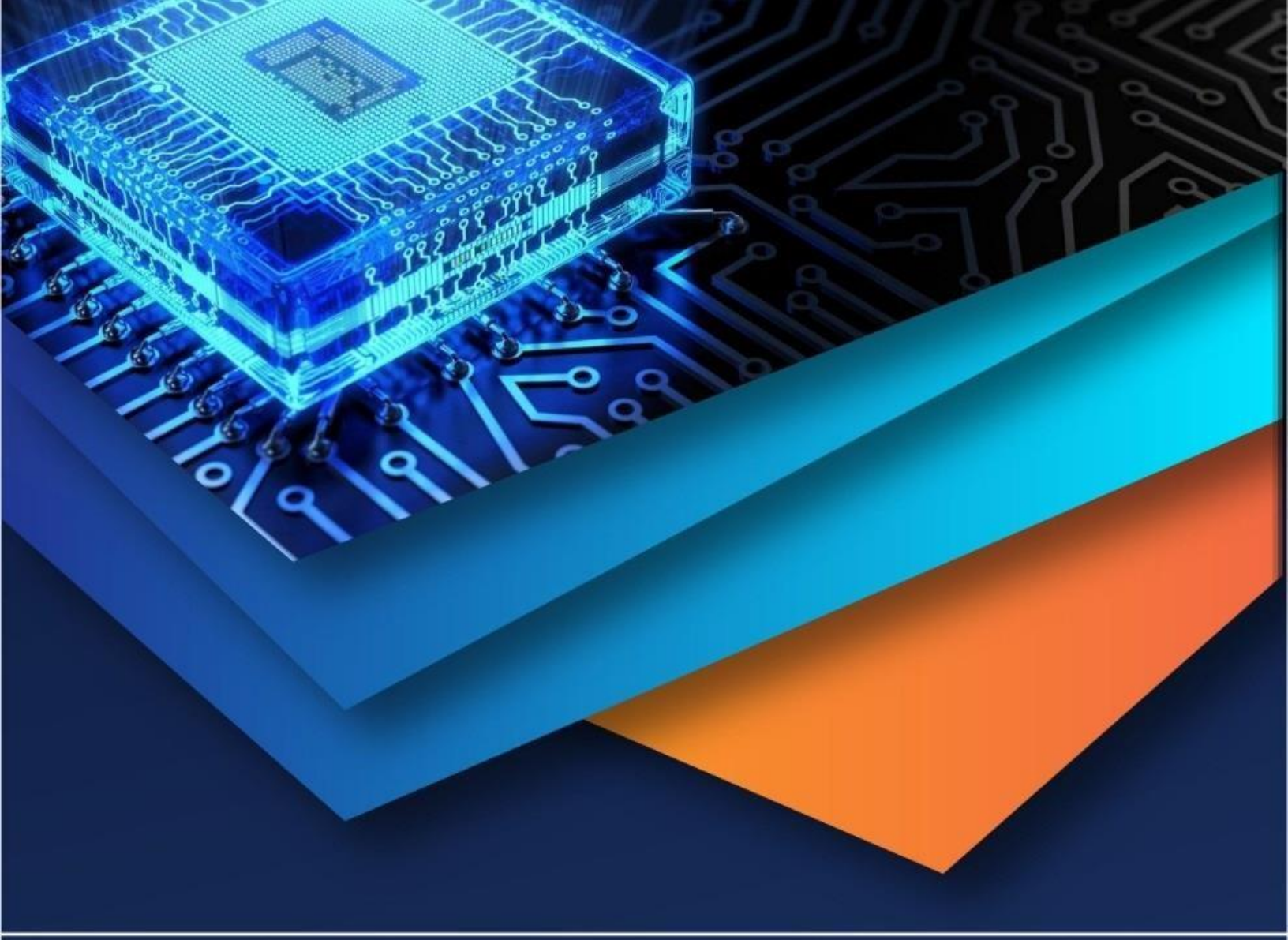
In this segment, we showcase the results derived from the experiments carried out to assess the effectiveness of our Hate Speech Detection Model. We evaluate the model's performance in discerning between instances of hate speech and non-hate speech, as per the metrics outlined in the methodology. We have attained an accuracy rate of 84, which already surpasses the performance of many existing systems. Efforts are underway to further enhance this accuracy, reflecting our ongoing commitment to model improvement.

V. CONCLUSION

In this study, we introduced an approach for the automated detection of hate tweets, leveraging machine learning through the utilization of TensorFlow, NLP techniques, and a Deep Neural Network (DNN). Our methodology includes the incorporation of both a bag of words and the TFIDF approach. Contrary to the logistic regression classifier mentioned previously, our model, implemented with TensorFlow and NLP, employs a DNN architecture for tweet classification into two categories. We adopted an 80-20 split, reserving 20% of the dataset for testing and using the remaining 80% for training. With the application of TensorFlow and NLP in our model, we achieved an accuracy of 84%, reflecting its robust performance. While this accuracy is slightly lower than the bag of words and TFIDF approach, the use of deep learning techniques opens avenues for further exploration and potential improvements. Future work includes experimenting with different classifiers and incorporating additional linguistic features to enhance the model's accuracy further.toward cultivating a highperformance culture that aligns individual success with organizational triumphs.

REFERENCES

[1] Design and development of a hate speech detector in social networks based on deep learning technologies D Benito Sánchez - 2019 - oa.upm.es
[2] [HTML] Token replacement-based data augmentation methods for hate speech detection KJ Madukwe, X Gao, B Xue
[3] SEMAR: An interface for Indonesian hate speech detection using machine learning UAN Rohmawati, SW Sihwi, DE Cahyani
[4] Fake news detection using recurrent neural network based on bidirectional LSTM and GloVe L Abualigah, YY Al-Ajlouni, MS Daoud, M Altalhi…- Social Network Analysis …, 2024 – Springer
[5] [HTML] Sarcasm detection over social media platforms using hybrid auto-encoder-based model DK Sharma, B Singh, S Agarwal, H Kim, R Sharma - Electronics, 2022 - mdpi.com
[6] A review on social spam detection: Challenges, open issues, and future directions S Rao, AK Verma, T Bhatia - Expert Systems with Applications, 2021 – Elsevier
[7] [HTML] Exploring the frontiers of deep learning and natural language processing: A comprehensive overview of key challenges and emerging trends
[8] W Khan, A Daud, K Khan, S Muhammad… - … Language Processing …, 2023 – Elsevier
[9] Breaking Barriers in Sentiment Analysis and Text Emotion Detection: Toward a Unified Assessment Framework ADL Languré, M Zareei - IEEE Access, 2023 - ieeexplore.ieee.org



10.22214/IJRASET

45.98

IMPACT FACTOR: 7.129

IMPACT FACTOR: 7.429

INTERNATIONAL JOURNAL FOR RESEARCH

IN APPLIED SCIENCE & ENGINEERING TECHNOLOGY

Call : 08813907089 (24*7 Support on Whatsapp)

ISSN No. : 2321-9653

iJRASET

International Journal for Research in Applied Science & Engineering Technology

IJRASET is indexed with Crossref for DOI-DOI : 10.22214

Website : www.ijraset.com, E-mail : ijraset@gmail.com

Certificate

It is here by certified that the paper ID : IJRASET59053, entitled

Hate Speech Detection

by

Ms. E. Krishnaveni

after review is found suitable and has been published in
Volume 12, Issue III, March 2024
in

International Journal for Research in Applied Science & Engineering Technology
(International Peer Reviewed and Refereed Journal)
Good luck for your future endeavors

Editor in Chief, iJRASET

ISRA Journal Impact Factor: 7.429

45.98
INDEX COPERNICUS

THOMSON REUTERS
Researcher ID: N-9681-2016

10.22214/IJRASET
doi crossref

TOGETHER WE REACH THE GOAL
SJIF 7.429

ISSN No. : 2321-9653

iJRASET

International Journal for Research in Applied Science & Engineering Technology

IJRASET is indexed with Crossref for DOI-DOI : 10.22214

Website : www.ijraset.com, E-mail : ijraset@gmail.com

ISRA Journal Impact Factor: **7.429**

45.98 INDEX COPERNICUS

THOMSON REUTERS Researcher ID: N-9681-2016

10.22214/IJRASET doi crossref

TOGETHER WE REACH THE GOAL SJIF 7.429

Certificate

It is here by certified that the paper ID : IJRASET59053, entitled

Hate Speech Detection

by

Dadireddy Bhuvaneswar Reddy

after review is found suitable and has been published in
Volume 12, Issue III, March 2024
in
International Journal for Research in Applied Science & Engineering Technology
(International Peer Reviewed and Refereed Journal)
Good luck for your future endeavors

Editor in Chief, **iJRASET**